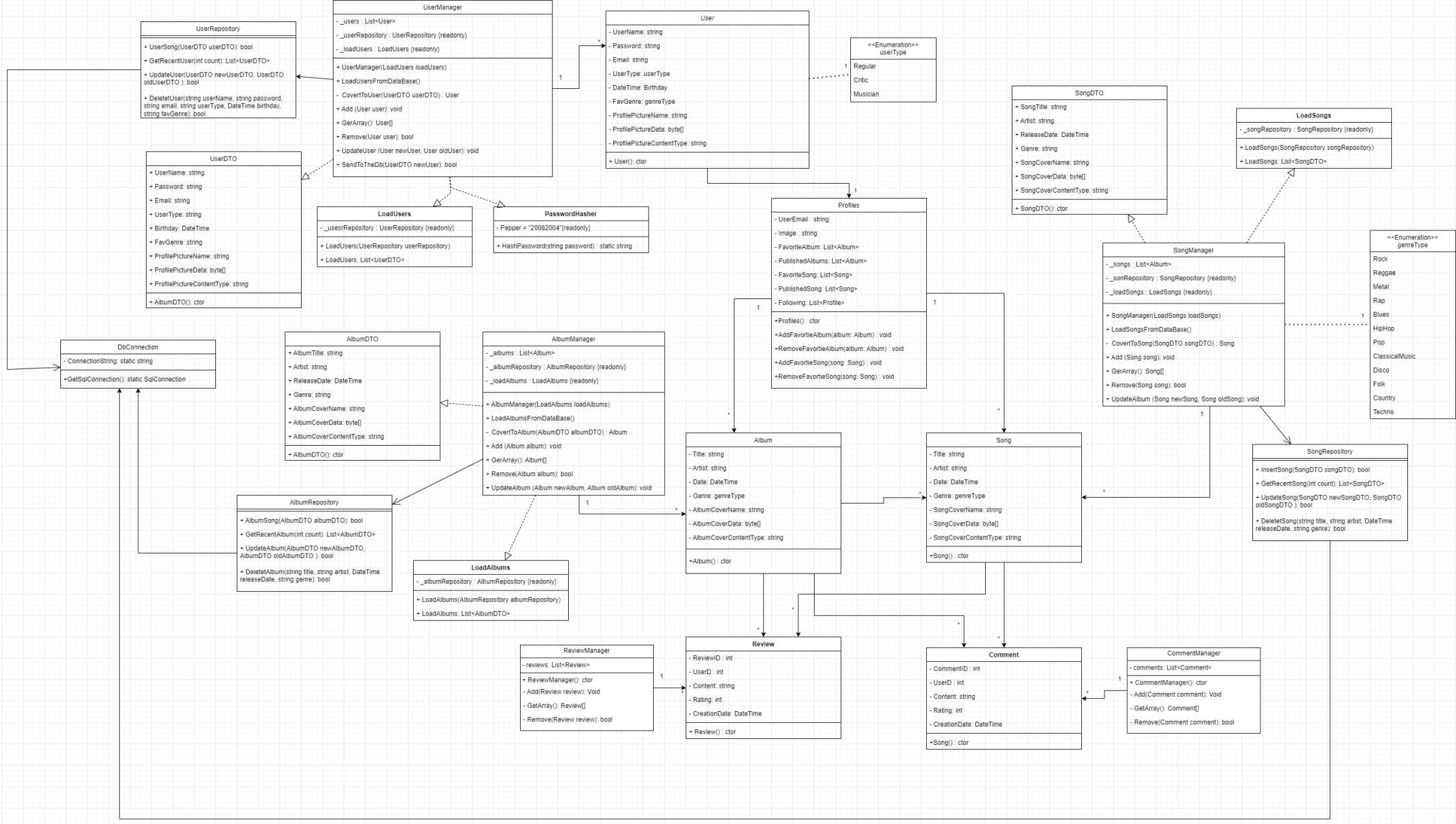

UserManager
- _users : List<User>
- _userRepository : UserRepository {readonly}
- _loadUsers : LoadUsers {readonly}
+ UserManager(LoadUsers loadUsers)
+ LoadUsersFromDataBase()
- CovertToUser(UserDTO userDTO) : User
+ Add (User user): void
+ GerArray(): User[]
+ Remove(User user): bool
+ UpdateUser (User newUser, User oldUser): void
+ SendToTheDb(UserDTO newUser): bool
UserRepository
+ UserSong(UserDTO userDTO): bool
+ GetRecentUser(int count): List<UserDTO>
+ UpdateUser(UserDTO newUserDTO, UserDTO oldUserDTO ): bool
+ DeletetUser(string userName, string password, string email, string userType, DateTime birthday, string favGenre): bool
User
- UserName: string
- Password: string
- Email: string
- UserType: userType
- DateTime: Birthday
- FavGenre: genreType
- ProfilePictureName: string
- ProfilePictureData: byte[]
- ProfilePictureContentType: string
+ User(): ctor
<<Enumeration>>
userType
Regular
Critic
Musician
UserDTO
+ UserName: string
+ Password: string
+ Email: string
+ UserType: string
+ Birthday: DateTime
+ FavGenre: string
+ ProfilePictureName: string
+ ProfilePictureData: byte[]
+ ProfilePictureContentType: string
+ AlbumDTO(): ctor
LoadUsers
- _usesrRepository : UserRepository {readonly}
+ LoadUsers(UserRepository userRepository)
+ LoadUsers: List<UserDTO>
PasswordHasher
- Pepper = "20082004"{readonly}
+ HashPassword(string password) : static string
Profiles
- UserEmail : string
- Image : string
- FavoriteAlbum: List<Album>
- PublishedAlbums: List<Album>
- FavoriteSong: List<Song>
- PublishedSong: List<Song>
- Following: List<Profile>
+Profiles() : ctor
+AddFavortieAlbum(album: Album) : void
+RemoveFavortieAlbum(album: Album) : void
+AddFavortieSong(song: Song) : void
+RemoveFavortieSong(song: Song) : void
SongDTO
+ SongTitle: string
+ Artist: string
+ ReleaseDate: DateTime
+ Genre: string
+ SongCoverName: string
+ SongCoverData: byte[]
+ SongCoverContentType: string
+ SongDTO(): ctor
LoadSongs
- _songRepository : SongRepository {readonly}
+ LoadSongs(SongRepository songRepository)
+ LoadSongs: List<SongDTO>
SongManager
- _songs : List<Album>
- _sonRepository : SongRepository {readonly}
- _loadSongs : LoadSongs {readonly}
+ SongManager(LoadSongs loadSongs)
+ LoadSongsFromDataBase()
- CovertToSong(SongDTO songDTO) : Song
+ Add (Song song): void
+ GerArray(): Song[]
+ Remove(Song song): bool
+ UpdateAlbum (Song newSong, Song oldSong): void
<<Enumeration>>
genreType
Rock
Reggae
Metal
Rap
Blues
HipHop
Pop
ClassicalMusic
Disco
Folk
Country
Techno
DbConnection
- ConnectionString: static string
+GetSqlConnection(): static SqlConnection
AlbumDTO
+ AlbumTitle: string
+ Artist: string
+ ReleaseDate: DateTime
+ Genre: string
+ AlbumCoverName: string
+ AlbumCoverData: byte[]
+ AlbumCoverContentType: string
+ AlbumDTO(): ctor
AlbumManager
- _albums : List<Album>
- _albumRepository : AlbumRepository {readonly}
- _loadAlbums : LoadAlbums {readonly}
+ AlbumManager(LoadAlbums loadAlbums)
+ LoadAlbumsFromDataBase()
- CovertToAlbum(AlbumDTO albumDTO) : Album
+ Add (Album album): void
+ GerArray(): Album[]
+ Remove(Album album): bool
+ UpdateAlbum (Album newAlbum, Album oldAlbum): void
Album
- Title: string
- Artist: string
- Date: DateTime
- Genre: genreType
- AlbumCoverName: string
- AlbumCoverData: byte[]
- AlbumCoverContentType: string
+Album() : ctor
Song
- Title: string
- Artist: string
- Date: DateTime
- Genre: genreType
- SongCoverName: string
- SongCoverData: byte[]
- SongCoverContentType: string
+Song() : ctor
SongRepository
+ InsertSong(SongDTO songDTO): bool
+ GetRecentSong(int count): List<SongDTO>
+ UpdateSong(SongDTO newSongDTO, SongDTO oldSongDTO ): bool
+ DeletetSong(string title, string artist, DateTime releaseDate, string genre): bool
AlbumRepository
+ AlbumSong(AlbumDTO albumDTO): bool
+ GetRecentAlbum(int count): List<AlbumDTO>
+ UpdateAlbum(AlbumDTO newAlbumDTO, AlbumDTO oldAlbumDTO ): bool
+ DeletetAlbum(string title, string artist, DateTime releaseDate, string genre): bool
LoadAlbums
- _albumRepository : AlbumRepository {readonly}
+ LoadAlbums(AlbumRepository albumRepository)
+ LoadAlbums: List<AlbumDTO>
ReviewManager
- reviews: List<Review>
+ ReviewManager(): ctor
- Add(Review review): Void
- GetArray(): Review[]
- Remove(Review review): bool
Review
- ReviewID : int
- UserD : int
- Content: string
- Rating: int
- CreationDate: DateTime
+ Review() : ctor
Comment
- CommentID : int
- UserD : int
- Content: string
- Rating: int
- CreationDate: DateTime
+Song() : ctor
CommentManager
- comments: List<Comment>
+ CommentManager(): ctor
- Add(Comment comment): Void
- GetArray(): Comment[]
- Remove(Comment comment): bool